Modern C++ — Review Quiz

A1. Does `auto` determine a variable's type at runtime? Explain briefly.

A2. Why can `int amount{3.14};` fail to compile while `int amount = 3.14;` is accepted? What is the practical implication?

A3. In a range-based `for` loop, what is the practical difference between `auto item` and `const auto& item`? When would you choose each?

A4. Why is `nullptr` preferable to `0` (or `NULL`) for a null pointer? Give one concrete reason involving overloaded functions.

A5. Does selecting the C++20 language standard in your project guarantee that every C++23 library feature (such as `std::print`) is available? Explain.

A6 (Predict the types). Under C++20, state the deduced type of each declaration:

```
auto count = 42;
auto ratio = 42.0;
auto one = {1};
auto two{1};
```

A7 (Identify the error). The following line is intended to initialize an integer variable with the value `2.75` truncated to `2`. Identify the problem and give a corrected version that makes the conversion explicit.

```
int quantity{2.75};
```

A8. What is the main safety benefit of `enum class` compared with an ordinary `enum`?

A9. What is the purpose of the semicolon in `if (init; condition)`? What is the scope of the variable declared in `init`?

A10. What does `inline` mean for a variable in modern C++? Does it force the compiler to substitute the value everywhere?

Answer Key

A1. No. `auto` asks the compiler to deduce a **static** type from the initializer at compile time. C++ remains statically typed; `auto` does not introduce runtime type determination.

A2. `int amount{3.14};` uses **list initialization**, which rejects narrowing conversions (double → int can lose the fractional part). `int amount = 3.14;` uses **copy initialization**, which permits the conversion (compilers commonly warn). Practical implication: braces help catch accidental loss of information; if truncation is intended, make it explicit with `static_cast<int>(3.14)` and document why.

A3. `auto item` creates a **copy** of each element; modifying `item` does not change the element in the range. `const auto& item` binds a **read-only reference** to the current element (no copy, cannot modify). Choose `auto` for small built-in types when a copy is harmless; choose `const auto&` for read-only iteration, especially over expensive-to-copy objects like strings or structs.

A4. `nullptr` has the dedicated type `std::nullptr_t` and clearly expresses pointer intent. In overload resolution, `show(0)` selects an `int` overload (since `0` is an integer literal), whereas `show(nullptr)` selects the pointer overload (`const char*`). Using `0` or `NULL` can therefore silently select the wrong overload.

A5. No. Selecting a language standard enables the corresponding language and library mode, but actual feature availability depends on the installed compiler toolset and standard-library implementation. A C++23 feature may still be unavailable if the toolset does not implement it.

A6.

```
auto count = 42;   // int
auto ratio = 42.0; // double
auto one = {1};    // std::initializer_list<int>
auto two{1};       // int (C++17 and later rules)
```

A7. The declaration `int quantity{2.75};` is ill-formed because list initialization prohibits the narrowing conversion from `double` to `int`. Corrected explicit version:

```
int quantity = static_cast<int>(2.75); // 2; document why truncation is safe
```

A8. `enum class` **scopes** its enumerator names (they do not leak into the surrounding scope) and **prevents implicit conversion to integers**, making different kinds of values harder to confuse accidentally. An ordinary `enum` allows name leakage and implicit integer conversion.

A9. The semicolon **separates the initializer statement from the condition**. The variable declared in the initializer is in scope for the condition and both branches of the `if`, but **not after** the complete `if` statement.

A10. For a variable, `inline` is a **linkage/ODR property**: it permits an equivalent definition to appear in multiple translation units (useful for constants defined in a header). It does **not** instruct the compiler to substitute the value into machine code everywhere; that is an optimization decision separate from the keyword.
